PYTHON ACTIVITY BASED LEARNING

Case Study Report

INTRODUCTION

This report presents ten Python Activity Based Learning case studies. Each case is organized with its problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python program, test cases, output screenshot, and conclusion. The material is compiled from the files supplied in the Python Activity Based Learning folder.

CONTENTS — TEN CASE STUDIES

Case 01. Smart Parking Fee Management
Case 02. Password Policy Validation Engine
Case 03. Campus Transport Allocation
Case 04. Smart Canteen Order and Stock Management
Case 05. Food Waste Monitoring System
Case 06. Sports Tournament Ranking System
Case 07. University Admission Decision Engine
Case 08. University Hostel Room Allocation
Case 09. Hackathon Team Formation and Ranking
Case 10. University Scholarship Allocation

Note: Text and code are taken from the submitted folder. Where a conclusion was not present as a separate source file, a concise conclusion has been written to complete the requested report structure.

CASE 01: Smart Parking Fee Management

1. Problem Statement

1. Problem statement
Develop a Python program to calculate parking fees based on vehicle category, entry time, exit time, overnight parking, daily limits, and penalty charges. The program must validate time inputs and display the final parking bill.

2. Objectives

2. Objectives
- Calculate parking duration and charges.
- Apply different rates for different vehicle categories.
- Handle overnight parking, daily limits, and penalties.
- Validate time inputs and display the final bill.

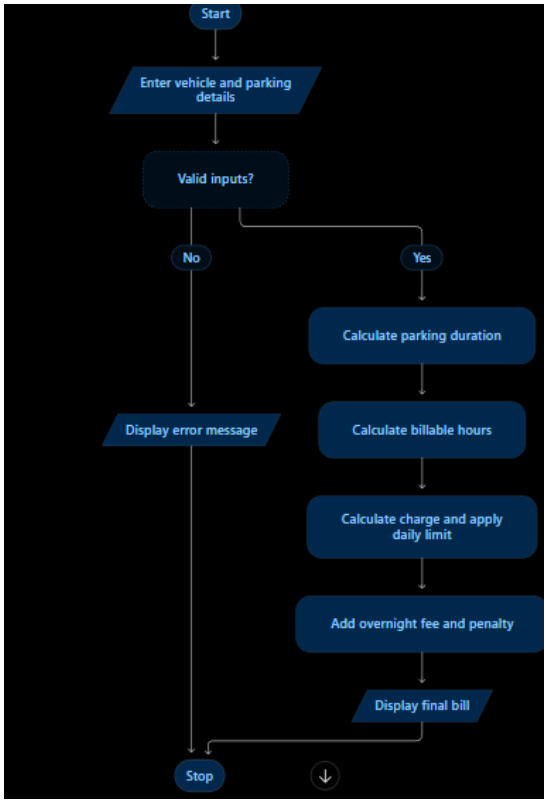
3. Assumptions

. Assumptions
These are sample rules for the program; they are not official parking rates.
Vehicle | Rate per hour | Daily limit
Bike | ₹10 | ₹100
Car | ₹20 | ₹200
Bus | ₹40 | ₹400
- Any partial hour is charged as a full hour.
- Overnight parking adds ₹50.
- A penalty of ₹100 applies if the user reports a lost ticket.
- Entry and exit times use the 24-hour HH:MM format.
- The user specifies whether the exit is on the next day.

4. Algorithm

Algorithm
1. Start.
2. Enter vehicle category, entry time, exit time, and overnight status.
3. Validate the vehicle category and time inputs.
4. Calculate the total parking duration in minutes.
5. Convert the duration into billable hours, rounding up partial hours.
6. Calculate the parking charge using the vehicle's hourly rate.
7. Apply the daily maximum limit.
8. Add the overnight fee and any applicable lost-ticket penalty.
9. Display the duration and final bill.
10. Stop.

5. Flowchart



6. Pseudocode

```
START

INPUT vehicle category
INPUT entry time and exit time
INPUT next-day exit status
INPUT lost-ticket status

VALIDATE all inputs

IF inputs are invalid
    DISPLAY error message
ELSE
    CALCULATE parking duration
    ROUND partial hours upward
    CALCULATE parking charge
    APPLY daily maximum limit

    IF overnight parking
        ADD overnight fee
    END IF

    IF ticket is lost
        ADD penalty
    END IF

    DISPLAY final bill
END IF

STOP
```

7. Python Source Code / Program

```
# Smart Parking Fee Management

rates = {
    "bike": 10,
    "car": 20,
    "bus": 40
}

daily_limits = {
    "bike": 100,
    "car": 200,
    "bus": 400
}

def read_time(message):
    while True:
        value = input(message).strip()
        parts = value.split(":")

        if len(parts) == 2 and all(p.isdigit() for p in parts):
            hour = int(parts[0])
            minute = int(parts[1])

            if 0 <= hour <= 23 and 0 <= minute <= 59:
                return hour * 60 + minute

        print("Invalid time! Enter time as HH:MM.")


vehicle = input("Vehicle (bike/car/bus): ").strip().lower()

if vehicle not in rates:
    print("Invalid vehicle category.")

else:
    entry = read_time("Entry time (HH:MM): ")
    exit_time = read_time("Exit time (HH:MM): ")

    next_day = input("Exit on next day? (yes/no): ").strip().lower()
    lost = input("Lost ticket? (yes/no): ").strip().lower()

    if next_day not in ["yes", "no"] or lost not in ["yes", "no"]:
        print("Invalid yes/no input.")

    elif next_day == "no" and exit_time <= entry:
        print("Invalid parking duration.")

    else:
        if next_day == "yes":
            duration = (1440 - entry) + exit_time
        else:
            duration = exit_time - entry

        hours = (duration + 59) // 60

        charge = hours * rates[vehicle]
        charge = min(charge, daily_limits[vehicle])

        overnight_fee = 50 if next_day == "yes" else 0
        penalty = 100 if lost == "yes" else 0

        total = charge + overnight_fee + penalty

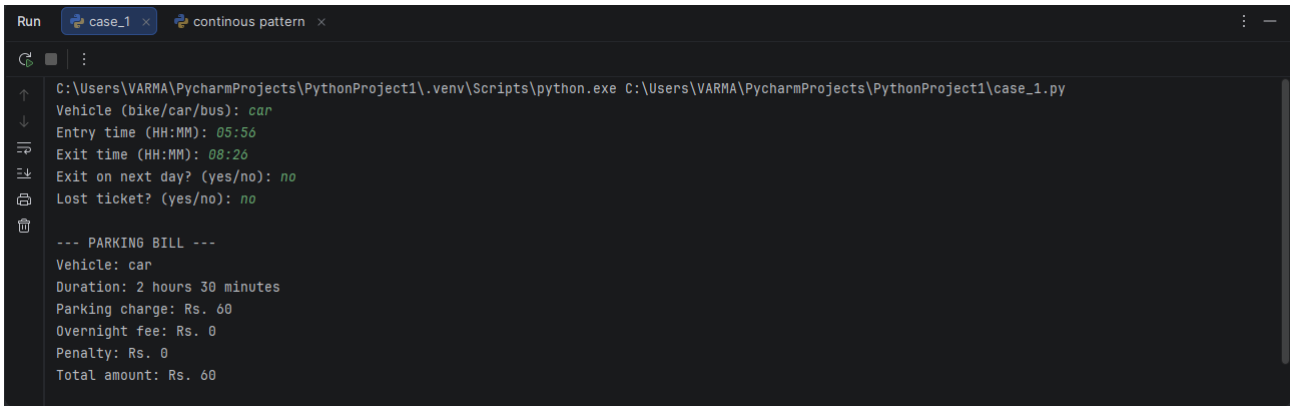
        print("\n--- PARKING BILL ---")
        print("Vehicle:", vehicle)
        print("Duration:", duration // 60, "hours",
              duration % 60, "minutes")
        print("Parking charge: Rs.", charge)
        print("Overnight fee: Rs.", overnight_fee)
        print("Penalty: Rs.", penalty)
        print("Total amount: Rs.", total)
```

8. Test Cases

Test cases

Test	Input details	Expected result
1	Bike, 09:00–11:00, same day, no penalty	₹20
2	Car, 09:00–10:30, same day, no penalty	₹40
3	Bike, 09:00–22:00, same day, no penalty	₹100 (daily limit)
4	Car, 22:00–02:00, next day, no penalty	₹130
5	Bus, 09:00–10:00, same day, lost ticket	₹140
6	Invalid entry time, such as 25:70	Ask for valid time again

9. Output Screenshot



10. Conclusion

This program calculates parking fees using vehicle type, duration, daily limits, overnight fees, and ticket penalties. Input validation and a clear bill help make parking-charge calculations consistent.

CASE 02: Password Policy Validation Engine

1. Problem Statement

Problem Statement
Passwords are used to protect user accounts and personal information. Weak passwords can make accounts vulnerable to unauthorized access. The Password Policy Validation Engine checks sample passwords against predefined rules, including minimum length, character categories, repeated characters, and prohibited substrings. It displays whether each password is valid and clearly explains any rules that it fails.

2. Objectives

Objectives
1.To develop a Python program to validate passwords based on predefined security rules.
2.To check the minimum password length of 8 characters.
3.To verify the presence of uppercase letters, lowercase letters, digits, and special characters.
4.To identify three or more identical consecutive characters in a password.
5.To detect prohibited substrings such as "password".
6.To display clear error messages explaining each failed condition.
7.To determine whether a password is valid or invalid based on the defined rules.

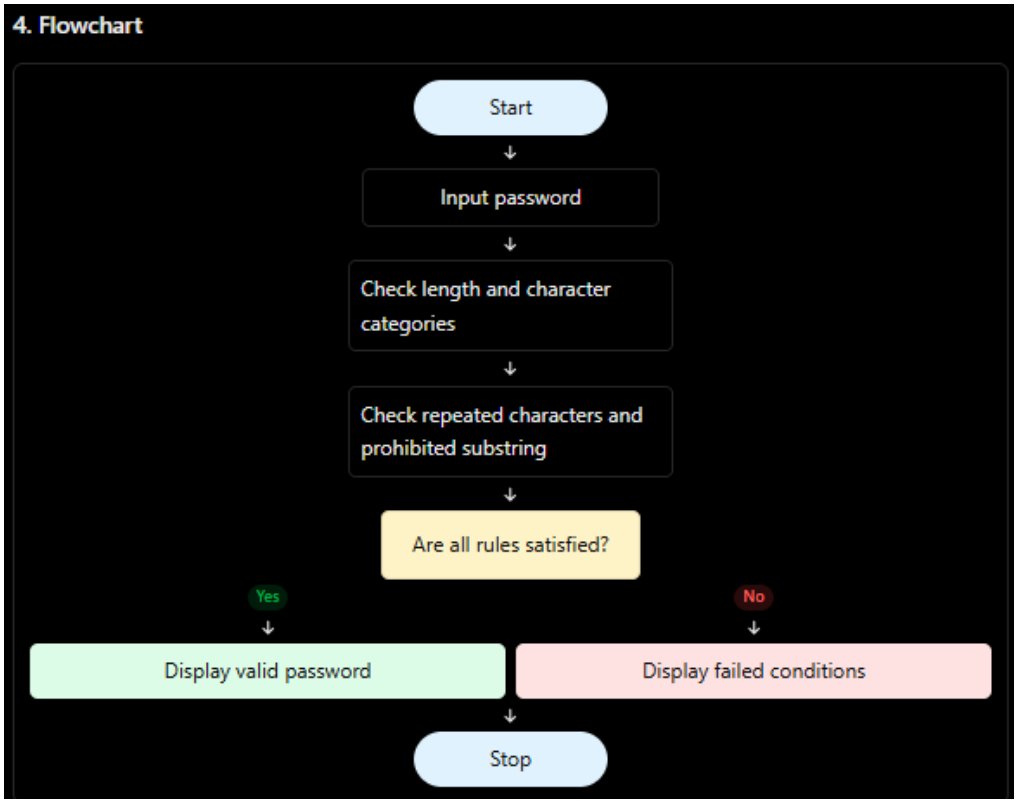
3. Assumptions

Assumptions
1. The minimum password length is 8 characters.
2. A valid password must contain at least one uppercase letter (A–Z).
3. A valid password must contain at least one lowercase letter (a–z).
4. A valid password must contain at least one digit (0–9).
5. A valid password must contain at least one special character, such as @, #, $, or !.
6. Three or more identical consecutive characters are prohibited.
7. The substring password is prohibited, regardless of letter case.
8. A password is accepted only when all the rules are satisfied.

4. Algorithm

Algorithm
1. Start the program.
2. Define the password validation rules.
3. Read a sample password from the user.
4. Check whether the password has at least 8 characters.
5. Check for uppercase letters, lowercase letters, digits, and special characters.
6. Check whether any character repeats three or more times consecutively.
7. Check whether the password contains the prohibited substring password, ignoring case.
8. Store and display all failed conditions.
9. If no rules fail, display “Password is Valid.” Otherwise, display “Password is Invalid.”
10. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT password
SET errors = empty list

IF length of password < 8
    ADD "Minimum length not satisfied" to errors

IF no uppercase letter exists
    ADD "Uppercase letter missing" to errors

IF no lowercase letter exists
    ADD "Lowercase letter missing" to errors

IF no digit exists
    ADD "Digit missing" to errors

IF no special character exists
    ADD "Special character missing" to errors

IF three or more identical consecutive characters exist
    ADD "Repeated characters not allowed" to errors

IF "password" occurs in password ignoring case
    ADD "Prohibited substring found" to errors

IF errors is empty
    DISPLAY "Password is Valid"
ELSE
    DISPLAY "Password is Invalid"
    DISPLAY all errors

STOP
```

7. Python Source Code / Program

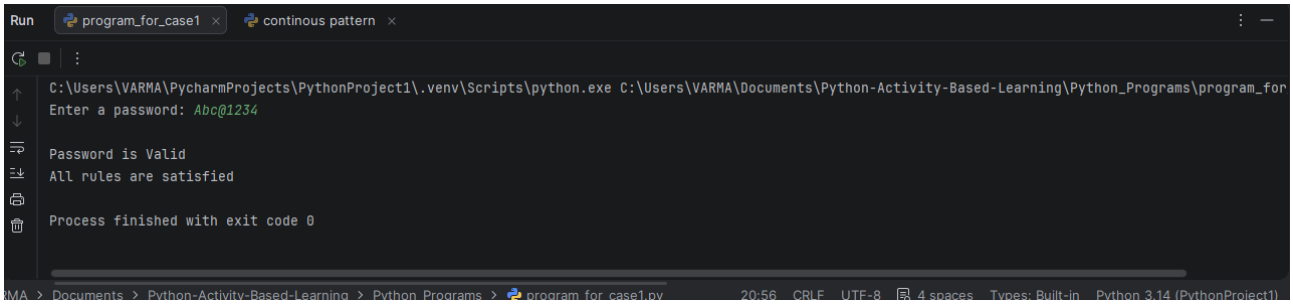
```
password = input("Enter a password: ")
errors = []
if len(password) < 8:
    errors.append("Password must contain at least 8 characters")
if not any(ch.isupper() for ch in password):
    errors.append("At least one uppercase letter is required")
if not any(ch.islower() for ch in password):
    errors.append("At least one lowercase letter is required")
if not any(ch.isdigit() for ch in password):
    errors.append("At least one digit is required")
special = "@#$!%&*?"
if not any(ch in special for ch in password):
    errors.append("At least one special character is required")
for i in range(len(password) - 2):
    if password[i] == password[i + 1] == password[i + 2]:
        errors.append("Three consecutive identical characters are not allowed")
        break
if "password" in password.lower():
    errors.append("The word 'password' is not allowed")
if len(errors) == 0:
    print("\nPassword is Valid")
    print("All rules are satisfied")
else:
    print("\nPassword is Invalid")
    print("Reasons:")
    for error in errors:
        print("-", error)
```

8. Test Cases

Test Cases

Test case	Input password	Expected result
TC01	Abc@1234	Valid
TC02	abc	Invalid — length and category rules fail
TC03	abcdefgh1@	Invalid — uppercase missing
TC04	ABCDEFG1@	Invalid — lowercase missing
TC05	Abcdefgh@	Invalid — digit missing
TC06	Abcdefg12	Invalid — special character missing
TC07	Abc@1111	Invalid — repeated consecutive characters
TC08	Password@1	Invalid — prohibited substring
TC09	Xy@98765	Valid
TC10	Aa@12345	Valid

9. Output Screenshot



10. Conclusion

This program checks a sample password against the defined security rules and reports the conditions that fail. It demonstrates how several independent checks can be combined to support safer password selection.

CASE 03: Campus Transport Allocation

1. Problem Statement

Problem Statement
Campus transport management requires allocating a sufficient number of buses to different routes based on passenger demand and bus capacity. If the number of buses is insufficient, some passengers may not get seats. This project develops a Python program to calculate the number of buses required for each route, determine the total buses needed, and identify the number of unserved passengers when only a limited number of buses are available.

2. Objectives

Objectives
1. To develop a Python program for campus bus allocation.
2. To calculate the number of buses required for each route based on passenger demand.
3. To consider the seating capacity of each bus.
4. To calculate the total number of buses required for all routes.
5. To identify the number of passengers who cannot be accommodated.
6. To display the allocation results clearly for efficient transport planning.

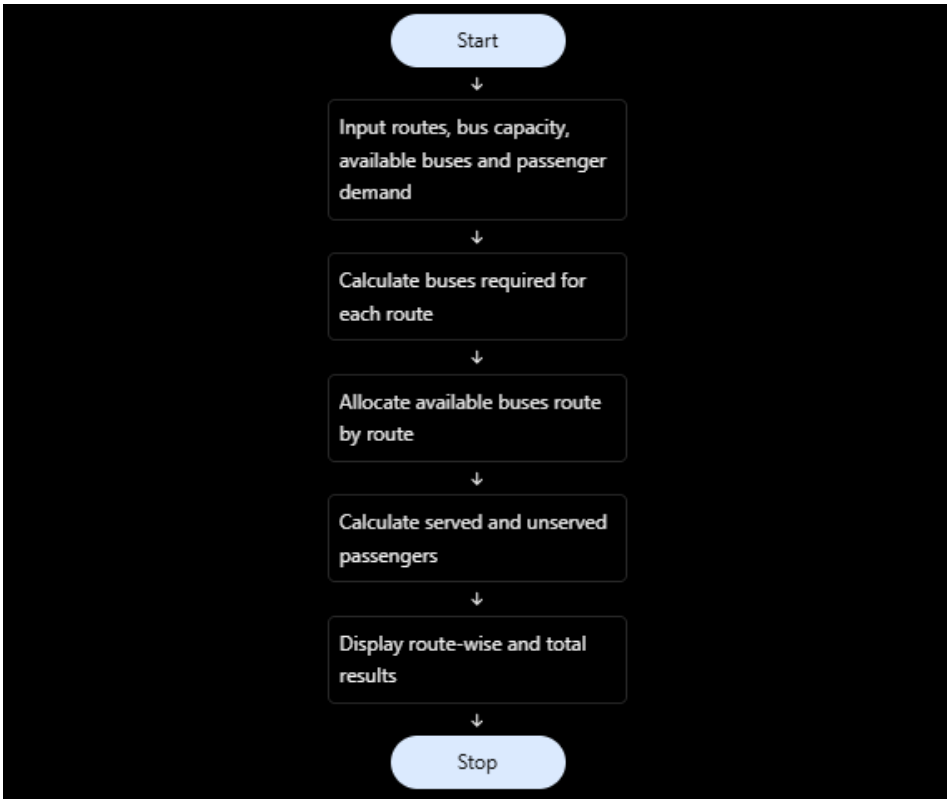
3. Assumptions

Assumptions
1. Each bus has a fixed seating capacity.
2. Passenger demand is known for every route.
3. All buses have the same seating capacity.
4. Each bus can operate on only one route for the allocation being calculated.
5. A bus cannot carry more passengers than its seating capacity.
6. The number of buses required for a route is rounded up to the next whole number.
7. If the available buses are insufficient, passengers exceeding the available capacity are considered unserved.
8. Passenger demand and bus capacity must be non-negative, and bus capacity must be greater than zero.

4. Algorithm

Algorithm
1. Start the program.
2. Input the number of routes, bus seating capacity, and total available buses.
3. Input the passenger demand for each route.
4. Calculate the buses required for each route using:
5. Calculate the total number of buses required for all routes.
6. Allocate available buses to routes in the order entered by the user.
7. Calculate the number of passengers who can be served on each route.
8. Calculate unserved passengers by subtracting served passengers from total demand.
9. Display the buses required, buses allocated, served passengers, and unserved passengers for each route.
10. Display the total buses required and total unserved passengers.
11. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT number_of_routes
INPUT bus_capacity
INPUT available_buses

SET total_required = 0
SET total_unserved = 0
SET remaining_buses = available_buses

FOR each route:
    INPUT passenger_demand

    required_buses = (passenger_demand + bus_capacity - 1)
                     // bus_capacity

    total_required = total_required + required_buses

    allocated_buses = MIN(required_buses, remaining_buses)

    remaining_buses = remaining_buses - allocated_buses

    served_passengers = MIN(passenger_demand,
                            allocated_buses * bus_capacity)

    unserved_passengers = passenger_demand - served_passengers

    total_unserved = total_unserved + unserved_passengers

    DISPLAY route details

DISPLAY total_required
DISPLAY total_unserved

STOP
```

7. Python Source Code / Program

```
print("Campus Transport Allocation")
capacity = int(input("Enter bus capacity: "))
buses = int(input("Enter available buses: "))
routes = int(input("Enter number of routes: "))
total_required = 0
total_unserved = 0
for i in range(1, routes + 1):
    passengers = int(input("Enter passengers for route " + str(i) + ": "))
    required = (passengers + capacity - 1) // capacity
    total_required += required
    if buses >= required:
        allocated = required
    else:
        allocated = buses

    buses = buses - allocated

    served = allocated * capacity

    if served > passengers:
        served = passengers

    unserved = passengers - served
    total_unserved += unserved

    print("Buses required:", required)
    print("Buses allocated:", allocated)
    print("Passengers served:", served)
    print("Unserved passengers:", unserved)

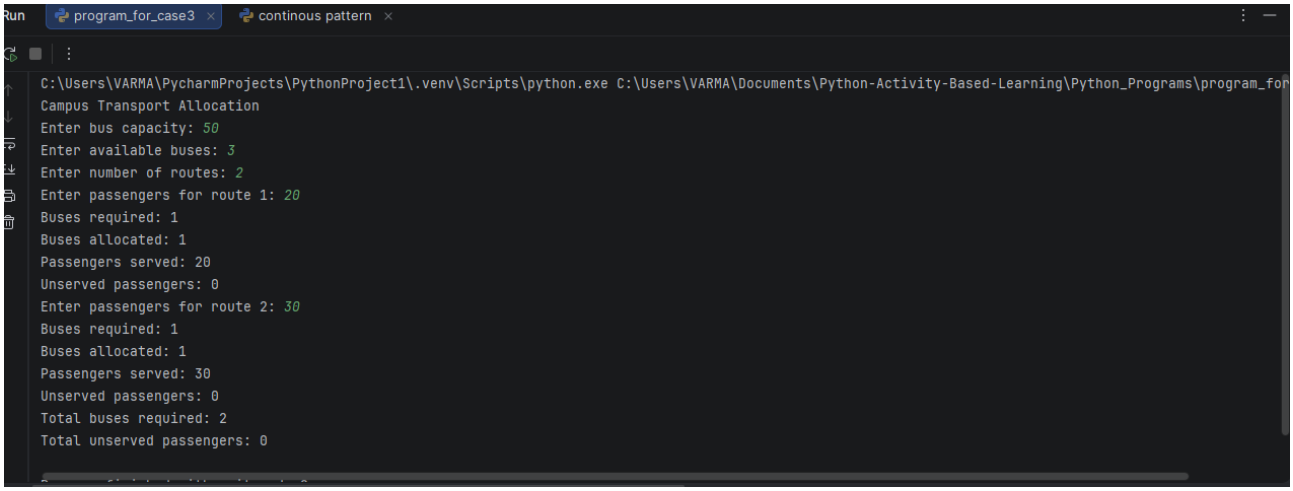
print("Total buses required:", total_required)
print("Total unserved passengers:", total_unserved)
```

8. Test Cases

Test Cases

Test case	Bus capacity	Available buses	Route demands	Expected result
TC01	40	6	70, 60, 50	6 buses required; 0 unserved
TC02	40	5	70, 60, 50	6 required; 10 unserved
TC03	50	4	20, 30, 40	3 required; 0 unserved
TC04	30	2	50, 40	4 required; 30 unserved
TC05	40	0	20, 30	2 required; 50 unserved
TC06	40	3	0, 0, 0	0 buses required; 0 unserved

9. Output Screenshot



10. Conclusion

This program estimates the buses needed for each campus route from passenger demand and seating capacity. It also identifies unserved passengers when the available fleet is insufficient.

CASE 04: Smart Canteen Order and Stock Management

1. Problem Statement

Problem Statement
A canteen needs a simple system to process customer orders, calculate bills, update available stock, and track daily sales. Manually managing orders and inventory can lead to billing errors and stock shortages. This project develops a Python program that accepts successive customer orders, checks product availability, rejects orders that exceed available stock, calculates bills, updates inventory after successful orders, and displays total daily sales and remaining stock.

2. Objectives

Objectives
1. To develop a Python program for canteen order and stock management.
2. To process multiple customer orders successively.
3. To calculate the bill based on item prices and quantities.
4. To check stock availability before confirming an order.
5. To reject orders when the requested quantity exceeds available stock.
6. To update inventory after each successful order.
7. To calculate total daily sales and display the remaining stock.

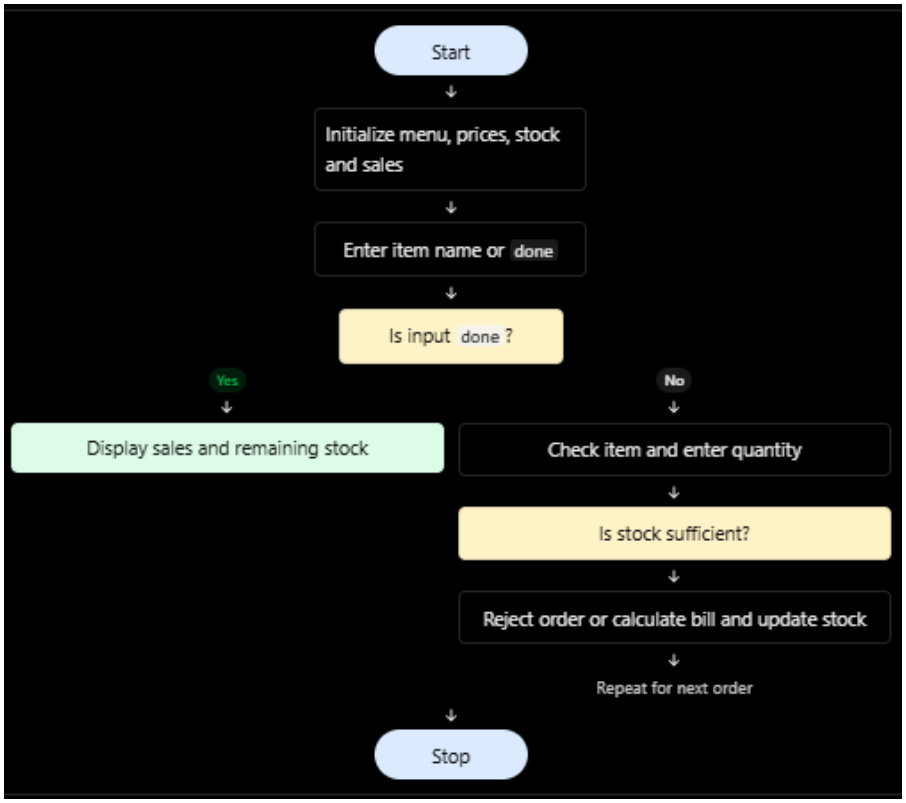
3. Assumptions

Assumptions
1. The canteen sells a fixed list of food items.
2. Each item has a fixed price and an initial stock quantity.
3. Customers select items using their names.
4. The requested quantity must be a positive whole number.
5. An order is accepted only if the requested quantity is available.
6. Stock is reduced only when an order is accepted.
7. Rejected orders do not contribute to daily sales.
8. The program ends when the user enters done instead of an item name.

4. Algorithm

Algorithm
1. Start the program.
2. Initialize the food items, prices, and available stock.
3. Set total daily sales to zero.
4. Ask the customer to enter a food item name.
5. If the customer enters done, stop accepting orders.
6. Check whether the item exists in the menu.
7. If the item exists, ask for the required quantity.
8. Check whether the quantity is valid and sufficient stock is available.
9. If stock is insufficient, reject the order without changing the stock.
10. Otherwise, calculate the bill and update the remaining stock.
11. Add the bill amount to total daily sales.
12. Repeat the process for the next order.
13. Display total daily sales and remaining stock for all items.
14. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INITIALIZE menu with item prices
INITIALIZE stock quantities
SET total_sales = 0

REPEAT
    INPUT item_name

    IF item_name = "done"
        EXIT LOOP

    IF item_name does not exist
        DISPLAY "Item not available"
        CONTINUE

    INPUT quantity

    IF quantity <= 0
        DISPLAY "Invalid quantity"
        CONTINUE

    IF quantity > available stock
        DISPLAY "Insufficient stock"
    ELSE
        bill = price × quantity
        stock = stock - quantity
        total_sales = total_sales + bill
        DISPLAY bill

UNTIL user enters "done"

DISPLAY total_sales
DISPLAY remaining stock of all items

STOP
```

7. Python Source Code / Program

```
print("Smart Canteen Management")

idli = 20
dosa = 15
tea = 30

sales = 0

while True:
    print("\n1. Idli - Rs.20 (Stock:", idli, ")")
    print("2. Dosa - Rs.40 (Stock:", dosa, ")")
    print("3. Tea - Rs.10 (Stock:", tea, ")")
    print("0. Finish orders")

    choice = int(input("Enter your choice: "))

    if choice == 0:
        break

    quantity = int(input("Enter quantity: "))

    if quantity <= 0:
        print("Invalid quantity!")

    elif choice == 1:
        if quantity <= idli:
            bill = quantity * 20
            idli -= quantity
            sales += bill
            print("Bill: Rs.", bill)
        else:
            print("Insufficient idli stock!")

    elif choice == 2:
        if quantity <= dosa:
            bill = quantity * 40
            dosa -= quantity
            sales += bill
            print("Bill: Rs.", bill)
        else:
            print("Insufficient dosa stock!")

    elif choice == 3:
        if quantity <= tea:
            bill = quantity * 10
            tea -= quantity
            sales += bill
            print("Bill: Rs.", bill)
        else:
            print("Insufficient tea stock!")

    else:
        print("Invalid choice!")

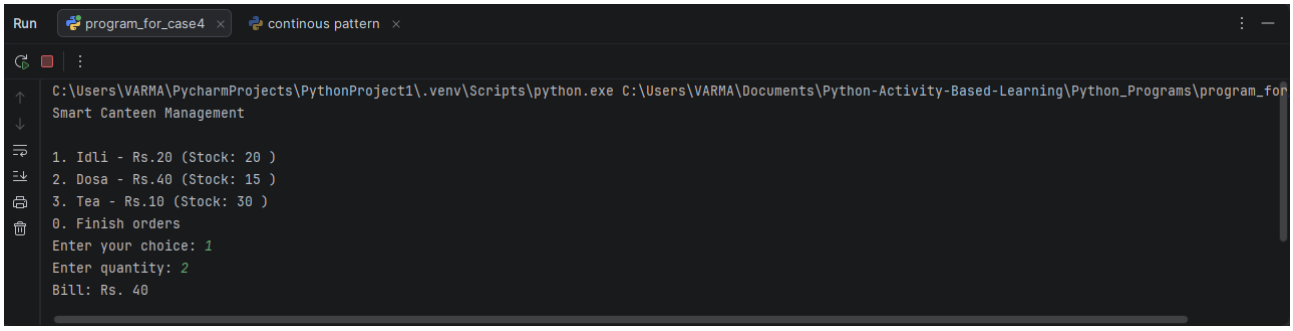
print("\n--- Daily Report ---")
print("Total Sales: Rs.", sales)
print("Remaining Idli:", idli)
print("Remaining Dosa:", dosa)
print("Remaining Tea:", tea)
```

8. Test Cases

Test Cases

Test case	Input / condition	Expected result
TC01	Order 2 idlis	Bill ₹40; stock decreases by 2
TC02	Order 3 dosas	Bill ₹120; stock decreases by 3
TC03	Order 50 teas	Rejected because stock is insufficient
TC04	Enter an unknown item, pizza	Item not available
TC05	Enter quantity 0	Invalid quantity
TC06	Place multiple successful orders	Daily sales show the sum of accepted bills
TC07	Enter done	Daily report and remaining stock are displayed

9. Output Screenshot



10. Conclusion

This program processes canteen orders, checks stock before accepting each order, updates inventory, calculates bills, and tracks daily sales. It helps reduce manual billing mistakes and stock-management confusion.

CASE 05: Food Waste Monitoring System

1. Problem Statement

Problem Statement
Food waste is a common problem in schools, colleges, hostels, and canteens. Preparing more food than needed leads to unnecessary waste and increased costs. The Food Waste Monitoring System compares the number of meals prepared, meals served, and leftover meals over several days. It calculates the food wastage percentage and identifies days when wastage exceeds an acceptable limit.

2. Objectives

Objectives
1. To develop a Python program to monitor food waste over several days.
2. To record the number of meals prepared and meals served each day.
3. To calculate the number of leftover meals.
4. To calculate the daily food wastage percentage.
5. To identify days when food wastage exceeds the acceptable limit.
6. To help improve food preparation and reduce unnecessary waste.

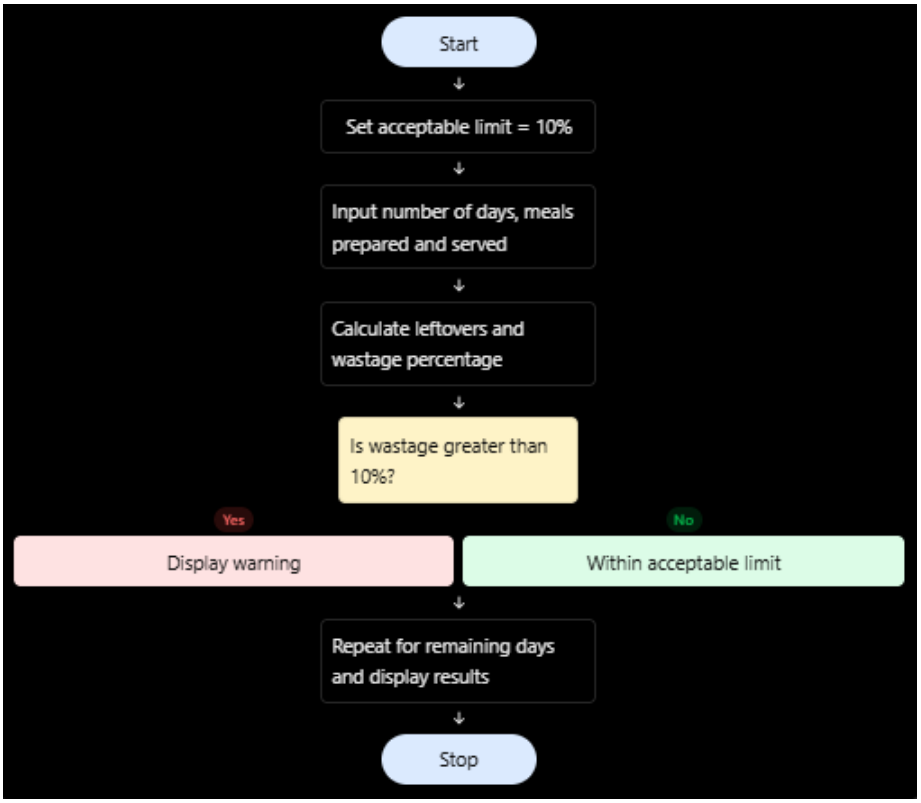
3. Assumptions

Assumptions
1. The system records data for multiple days.
2. The number of meals prepared and served is a non-negative whole number.
3. Meals served cannot exceed meals prepared.
4. Leftover meals are calculated as meals prepared minus meals served.
5. The acceptable food wastage limit is assumed to be 10%.
6. Wastage percentage is calculated using the number of meals prepared.
7. If wastage exceeds 10%, the day is marked as exceeding the acceptable limit.

4. Algorithm

Algorithm
1. Start the program.
2. Set the acceptable wastage limit to 10%.
3. Enter the number of days to monitor.
4. For each day, input the number of meals prepared and meals served.
5. Calculate leftover meals by subtracting meals served from meals prepared.
6. Calculate the wastage percentage using the formula:
7. Compare the wastage percentage with the acceptable limit.
8. If wastage exceeds 10%, display a warning for that day.
9. Otherwise, display that wastage is within the acceptable limit.
10. Repeat for all days.
11. Display the results for each day.
12. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET limit = 10

INPUT number_of_days

FOR each day:
    INPUT meals_prepared
    INPUT meals_served

    leftovers = meals_prepared - meals_served

    IF meals_prepared > 0:
        wastage = (leftovers / meals_prepared) * 100
    ELSE:
        wastage = 0

    DISPLAY leftovers
    DISPLAY wastage percentage

    IF wastage > limit:
        DISPLAY "Wastage exceeds acceptable limit"
    ELSE:
        DISPLAY "Wastage is within acceptable limit"

STOP
```

7. Python Source Code / Program

```
print("Food Waste Monitoring System")
days = int(input("Enter number of days: "))
total_waste = 0
for i in range(1, days + 1):
    print("\nDay", i)
    prepared = int(input("Meals prepared: "))
    served = int(input("Meals served: "))
    leftover = prepared - served
    total_waste += leftover
    if prepared > 0:
        percentage = leftover / prepared * 100
    else:
        percentage = 0

    print("Leftover meals:", leftover)
    print("Wastage percentage:", round(percentage, 2), "%")

    if percentage > 10:
        print("High food waste!")
    else:
        print("Food waste is acceptable.")

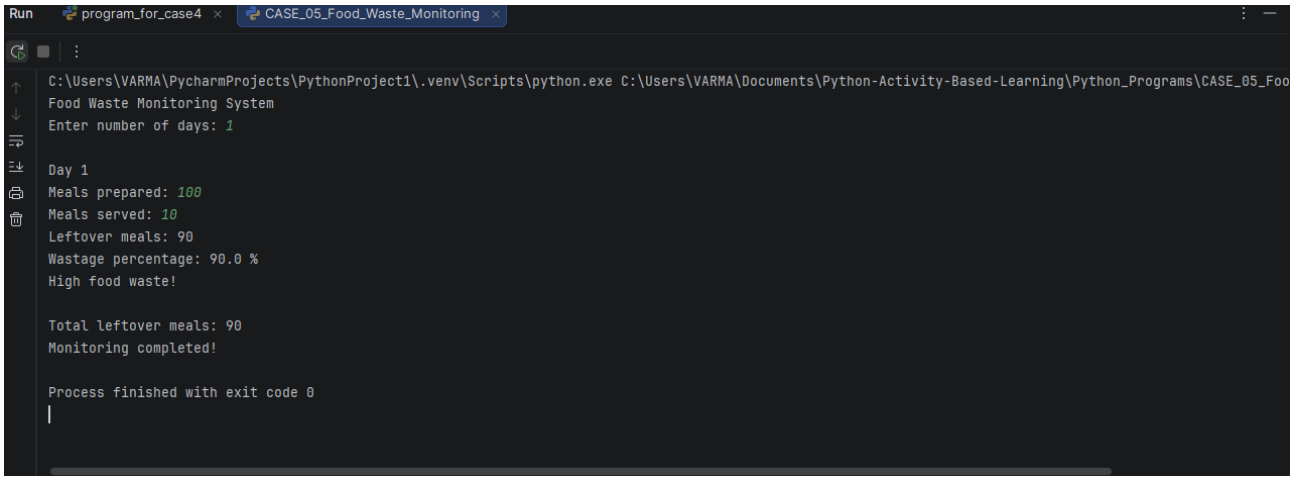
print("\nTotal leftover meals:", total_waste)
print("Monitoring completed!")
```

8. Test Cases

Test Cases

Test case	Meals prepared	Meals served	Leftovers	Wastage	Expected result
TC01	100	95	5	5%	Within limit
TC02	100	80	20	20%	Exceeds limit
TC03	200	180	20	10%	Within limit
TC04	50	40	10	20%	Exceeds limit
TC05	100	100	0	0%	Within limit
TC06	0	0	0	0%	Within limit

9. Output Screenshot



10. Conclusion

This program calculates leftovers and wastage percentages for each monitored day. Comparing the result with the chosen 10% limit helps identify days that may need improved meal planning.

CASE 06: Sports Tournament Ranking System

1. Problem Statement

Problem Statement
In sports tournaments, teams must be ranked based on their match results and scores. Calculating rankings manually can lead to errors, especially when teams have equal points. This project develops a Python program to record team results, calculate total points and tie-break values, and display the final rankings according to a predefined competition policy.

2. Objectives

Objectives
1. To develop a Python program for ranking teams in a sports tournament.
2. To store team names and match results.
3. To calculate total points earned by each team.
4. To calculate tie-break values using scores.
5. To rank teams according to the competition rules.
6. To display the final tournament standings.

3. Assumptions

Assumptions
1. Each team plays a fixed number of matches.
2. A win earns 3 points, a draw earns 1 point, and a loss earns 0 points.
3. Each match records the scores of both teams.
4. The tie-break value is calculated as total scores scored minus total scores conceded.
5. Teams are ranked by total points first, then by tie-break value.
6. If both values are equal, the teams share the same ranking position.
7. Match scores cannot be negative.

4. Algorithm

Algorithm
1. Start the program.
2. Input the number of teams.
3. Enter each team's name and initialize its points and scores to zero.
4. Input the number of matches.
5. For each match, enter the two team names and their scores.
6. Award 3 points to the winner or 1 point to each team in a draw.
7. Update each team's total score and scores conceded.
8. Calculate the tie-break value as scores scored minus scores conceded.
9. Sort teams by total points in descending order, followed by tie-break value in descending order.
10. Display the final rankings, points, and tie-break values.
11. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT team names
INITIALIZE points and scores to zero

INPUT number of matches

FOR each match:
    INPUT team1, team2
    INPUT score1, score2

    UPDATE scores scored and conceded

    IF score1 > score2:
        team1 gets 3 points
    ELSE IF score2 > score1:
        team2 gets 3 points
    ELSE:
        both teams get 1 point

CALCULATE tie-break value for each team

SORT teams by:
    1. Total points (descending)
    2. Tie-break value (descending)

DISPLAY team rankings and points

STOP
```

7. Python Source Code / Program

```
print("Sports Tournament Ranking System")

teams = []
points = []
scores = []

n = int(input("Enter number of teams: "))

for i in range(n):
    name = input("Enter team name: ")
    teams.append(name)
    points.append(0)
    scores.append(0)

matches = int(input("Enter number of matches: "))

for i in range(matches):
    print("\nMatch", i + 1)

    t1 = int(input("Enter first team number (1 to n): ")) - 1
    t2 = int(input("Enter second team number (1 to n): ")) - 1

    s1 = int(input("Enter first team score: "))
    s2 = int(input("Enter second team score: "))

    scores[t1] += s1 - s2
    scores[t2] += s2 - s1

    if s1 > s2:
        points[t1] += 3
    elif s2 > s1:
        points[t2] += 3
    else:
        points[t1] += 1
        points[t2] += 1

print("\n--- FINAL RANKINGS ---")

for i in range(n):
    for j in range(i + 1, n):
        if points[j] > points[i] or (
            points[j] == points[i] and scores[j] > scores[i]
        ):
            points[i], points[j] = points[j], points[i]
            scores[i], scores[j] = scores[j], scores[i]
            teams[i], teams[j] = teams[j], teams[i]

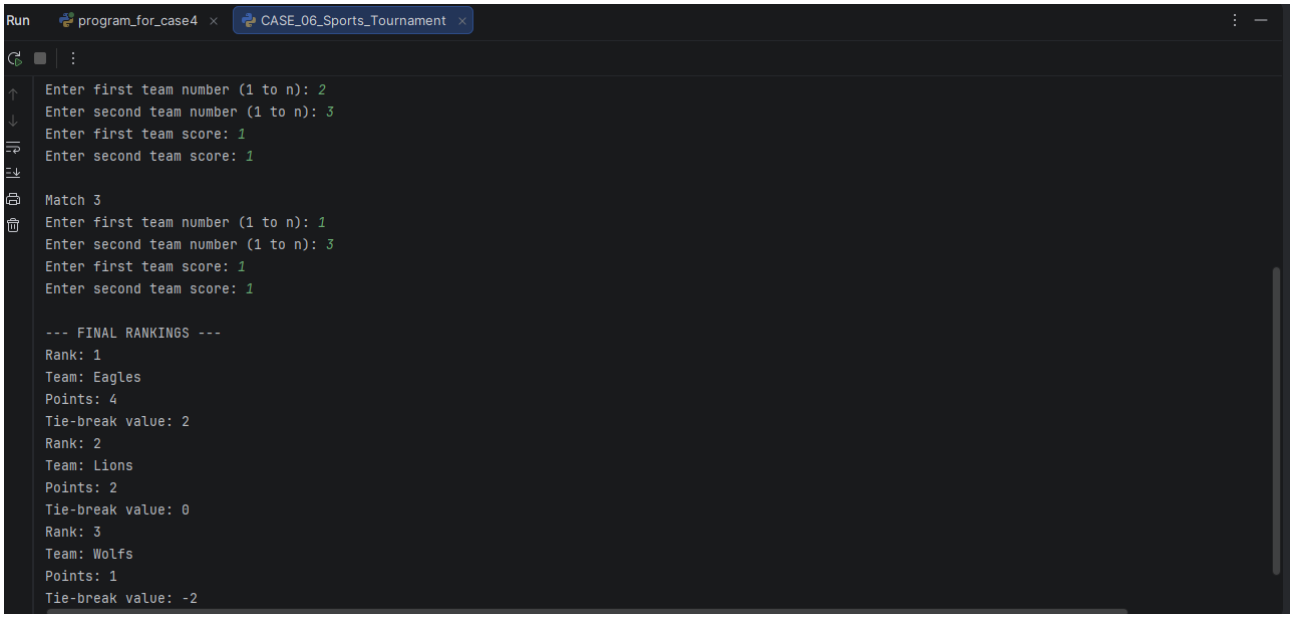
for i in range(n):
    print("Rank:", i + 1)
    print("Team:", teams[i])
    print("Points:", points[i])
    print("Tie-break value:", scores[i])
```

8. Test Cases

Test Cases

Test case	Match result	Expected result
TC01	Team A 3–1 Team B	A gets 3 points; B gets 0
TC02	Team A 2–2 Team B	Both get 1 point
TC03	Team B 4–1 Team C	B gets 3 points; C gets 0
TC04	Two teams have equal points but different tie-break values	Higher tie-break value ranks first
TC05	All matches completed	Final ranking is displayed

9. Output Screenshot



10. Conclusion

This program awards points from match results, calculates score-based tie-break values, and sorts teams into a final ranking. It provides a consistent way to prepare tournament standings.

CASE 07: University Admission Decision Engine

1. Problem Statement

Problem Statement
Universities receive applications from many students for different courses. Checking every application manually takes time and may lead to errors. The University Admission Decision Engine automates this process by checking subject eligibility, aggregate marks, and course requirements. It then identifies eligible applicants and generates a ranked list to support admission decisions.

2. Objectives

Objective
To develop a Python program that evaluates students for university admission based on subject eligibility, aggregate marks, and course-specific minimum marks. The program selects eligible candidates and ranks them according to their aggregate marks, using a tie-breaking rule when candidates have equal marks.

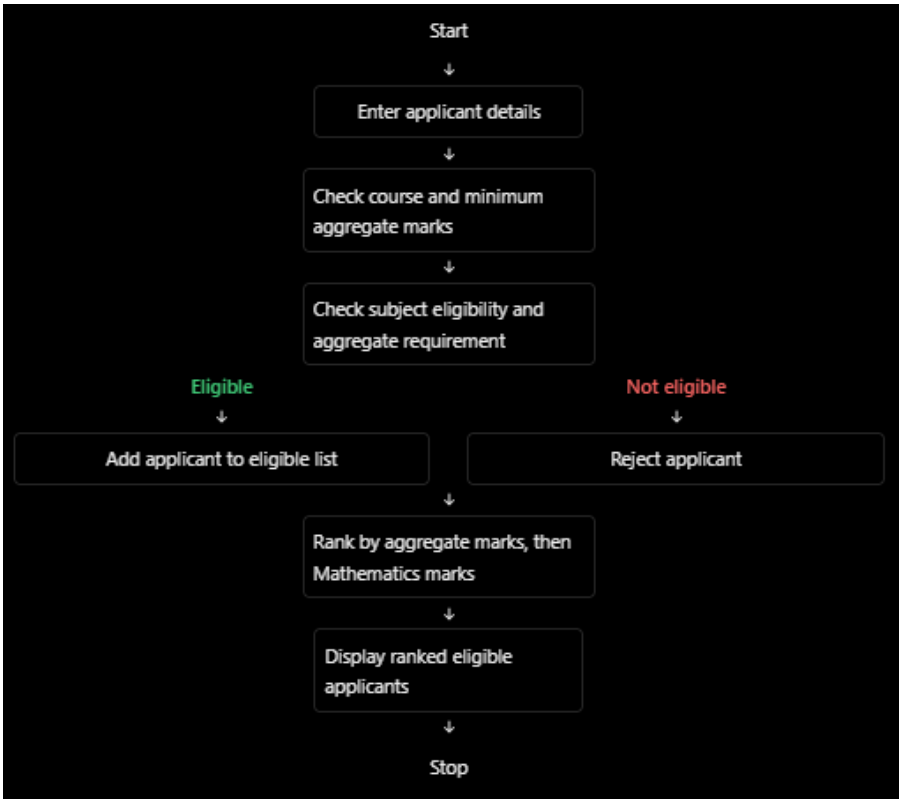
3. Assumptions

Assumptions
1. Each applicant has a name, subject eligibility status, and aggregate percentage.
2. The university offers two courses: CSE and ECE.
3. The minimum aggregate percentage is 75% for CSE and 65% for ECE.
4. A candidate must meet the subject eligibility requirements for the selected course.
5. Candidates with higher aggregate marks receive higher ranks.
6. If two candidates have the same aggregate marks, the candidate with higher Mathematics marks gets the higher rank.
7. Only eligible candidates are included in the final ranking.
8. The program assumes the entered marks and eligibility information are valid.

4. Algorithm

Algorithm
1. Start the program.
2. Read the number of applicants.
3. For each applicant, enter their name, course, subject eligibility, aggregate percentage, and Mathematics marks.
4. Check the minimum aggregate requirement for the selected course.
5. If the applicant satisfies both subject eligibility and the aggregate requirement, add them to the eligible list.
6. Arrange eligible applicants in descending order of aggregate marks.
7. If aggregate marks are equal, rank the applicant with higher Mathematics marks first.
8. Display the final ranked list of eligible applicants.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT number of applicants
CREATE an empty eligible list

FOR each applicant:
    INPUT name
    INPUT course
    INPUT subject eligibility
    INPUT aggregate marks
    INPUT Mathematics marks

    IF course is CSE:
        SET minimum marks to 75
    ELSE IF course is ECE:
        SET minimum marks to 65

    IF subject eligible AND aggregate >= minimum marks:
        ADD applicant to eligible list

SORT eligible applicants by:
    1. Aggregate marks in descending order
    2. Mathematics marks in descending order

DISPLAY ranked eligible applicants

STOP
```

7. Python Source Code / Program

```
print("University Admission Decision Engine")

eligible = []

n = int(input("Enter number of applicants: "))

for i in range(n):
    print("\nApplicant", i + 1)

    name = input("Enter name: ")
    course = input("Enter course (CSE/ECE): ").upper()
    subject = input("Subject eligible? (yes/no): ").lower()
    aggregate = float(input("Enter aggregate percentage: "))
    maths = float(input("Enter Mathematics marks: "))

    if course == "CSE":
        minimum = 75
    elif course == "ECE":
        minimum = 65
    else:
        print("Invalid course!")
        continue

    if subject == "yes" and aggregate >= minimum:
        eligible.append([name, course, aggregate, maths])
        print("Eligible for admission")
    else:
        print("Not eligible for admission")

# Sort by aggregate marks, then Mathematics marks
for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        if (eligible[j][2] > eligible[i][2] or
            (eligible[j][2] == eligible[i][2] and
             eligible[j][3] > eligible[i][3])):

            eligible[i], eligible[j] = eligible[j], eligible[i]

print("\n--- FINAL ADMISSION RANKING ---")

if len(eligible) == 0:
    print("No eligible applicants found.")
else:
    for i in range(len(eligible)):
        print("\nRank:", i + 1)
        print("Name:", eligible[i][0])
        print("Course:", eligible[i][1])
        print("Aggregate:", eligible[i][2])
        print("Mathematics marks:", eligible[i][3])
```

8. Test Cases

Test Cases
Use these test cases to check whether the program works correctly.

Test case	Course	Subject eligible?	Aggregate	Expected result
1	CSE	yes	90%	Eligible
2	CSE	yes	70%	Not eligible
3	ECE	yes	68%	Eligible
4	ECE	no	85%	Not eligible
5	CSE	yes	90%	Eligible; Mathematics marks break the tie

9. Output Screenshot



10. Conclusion

This program checks applicants against course-specific admission requirements and ranks eligible candidates using the stated rules. It supports more consistent screening and tie-breaking.

CASE 08: University Hostel Room Allocation

1. Problem Statement

Problem Statement
Universities accommodate many students in hostels, and assigning rooms manually can cause confusion, duplicate allocations, and capacity issues. A Hostel Room Allocation System automates the allocation process by checking student eligibility, room capacity, and grouping requirements. It identifies duplicate assignments and calculates the remaining unallocated student demand, helping hostel administrators manage rooms efficiently.

2. Objectives

Objectives
To develop a Python program that allocates students to hostel rooms based on room capacity, student eligibility, and grouping constraints. The program also detects duplicate student assignments and calculates the number of students who could not be allocated a room.

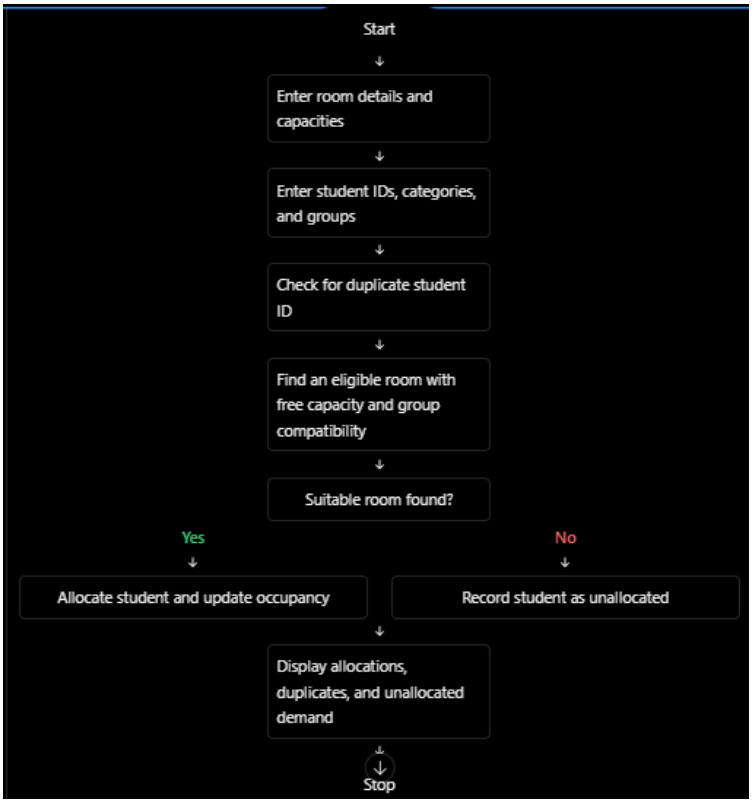
3. Assumptions

Assumptions
1. Each student has a unique student ID and a name.
2. Each hostel room has a room number, capacity, and eligibility category.
3. Students can be assigned only to rooms for which they are eligible.
4. A room cannot exceed its maximum capacity.
5. Students belonging to the same specified group should be assigned to the same room whenever possible.
6. A student cannot be assigned to more than one room.
7. If a student cannot be allocated a room, they are counted as unallocated.
8. All student details and room capacities are entered correctly.

4. Algorithm

Algorithm
1. Start the program.
2. Enter the number of hostel rooms.
3. Enter each room's number, capacity, and category.
4. Enter the number of students.
5. Enter each student's ID, name, category, and group.
6. Check whether the student ID has already been entered.
7. Find a room with matching eligibility, available capacity, and the required group.
8. If a suitable room is found, allocate the student and update the room's occupancy.
9. If no suitable room is available, mark the student as unallocated.
10. Display room allocations, duplicate assignments, and total unallocated students.
11. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT room details
INPUT student details

CREATE empty lists for allocations and duplicates

FOR each student:
    IF student ID already exists:
        RECORD duplicate assignment
    ELSE:
        FIND a room that:
            matches student category
            has available capacity
            satisfies grouping constraints

        IF a suitable room is found:
            ALLOCATE student to that room
            UPDATE room occupancy
        ELSE:
            COUNT student as unallocated

DISPLAY room allocations
DISPLAY duplicate assignments
DISPLAY total unallocated students

STOP
```

7. Python Source Code / Program

```
print("University Hostel Room Allocation")

rooms = []
allocated = []
duplicates = []
unallocated = []
seen_ids = []
group_rooms = {}

n = int(input("Enter number of rooms: "))

for i in range(n):
    print("\nRoom", i + 1)
    number = input("Enter room number: ")
    capacity = int(input("Enter room capacity: "))
    category = input("Enter category (Boys/Girls): ").lower()

    rooms.append({
        "number": number,
        "capacity": capacity,
        "category": category,
        "students": []
    })

s = int(input("\nEnter number of students: "))

for i in range(s):
    print("\nStudent", i + 1)

    sid = input("Enter student ID: ")
    name = input("Enter student name: ")
    category = input("Enter category (Boys/Girls): ").lower()
    group = input("Enter group name: ")

    if sid in seen_ids:
        print("Duplicate student ID!")
        duplicates.append(sid)
        continue

    seen_ids.append(sid)
    assigned = False

    for room in rooms:
        if group in group_rooms:
            if room["number"] != group_rooms[group]:
                continue

        if (room["category"] == category and
            len(room["students"]) < room["capacity"]):

            room["students"].append(name)
            allocated.append(name)

            if group not in group_rooms:
                group_rooms[group] = room["number"]

            print("Allocated to room:", room["number"])
            assigned = True
            break

    if not assigned:
        print("No suitable room available.")
        unallocated.append(name)

print("\n--- FINAL HOSTEL REPORT ---")

for room in rooms:
    print("\nRoom:", room["number"])
    print("Category:", room["category"])
    print("Capacity:", room["capacity"])
    print("Students:", room["students"])
    print("Occupied:", len(room["students"]))

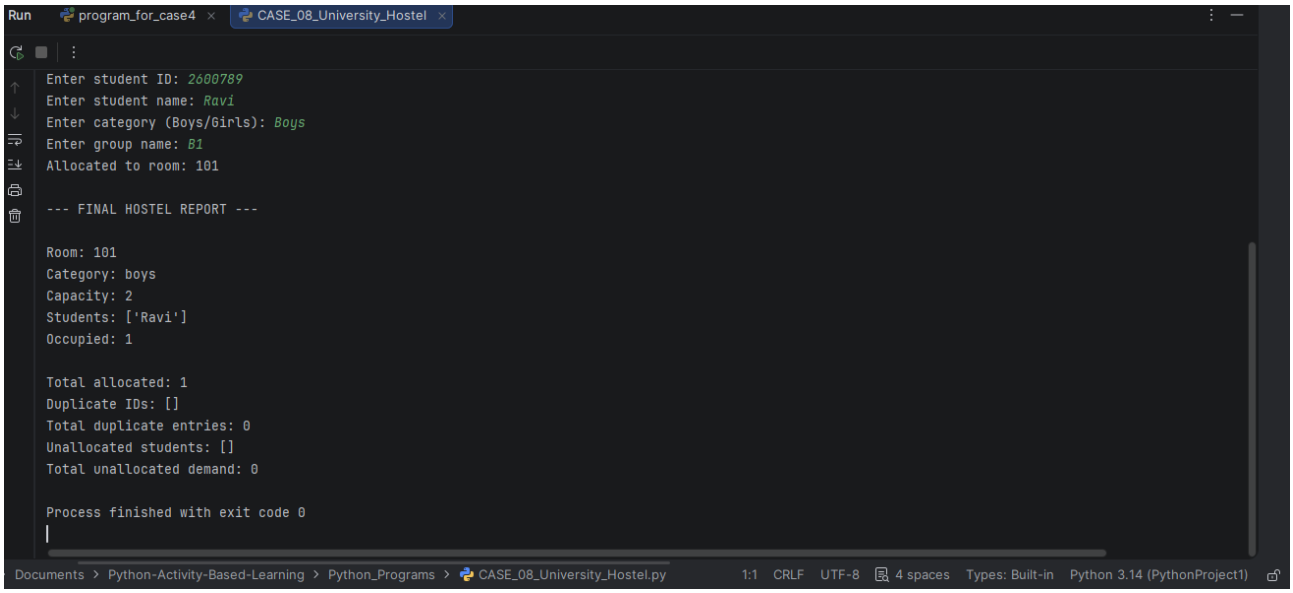
print("\nTotal allocated:", len(allocated))
print("Duplicate IDs:", duplicates)
```

```
print("Total duplicate entries:", len(duplicates))
print("Unallocated students:", unallocated)
print("Total unallocated demand:", len(unallocated))
```

8. Test Cases

Test Cases
Test case | Input condition | Expected result
1 | Eligible student; room has space | Student allocated
2 | Room is full | Student unallocated
3 | Student category does not match room | Student unallocated
4 | Same student ID entered twice | Duplicate detected
5 | Group's assigned room is full | Student unallocated
6 | No eligible rooms available | All affected students unallocated

9. Output Screenshot



10. Conclusion

This program assigns students to rooms while considering eligibility, room capacity, grouping rules, and duplicate IDs. It also identifies students who could not be allocated a room.

CASE 09: Hackathon Team Formation and Ranking

1. Problem Statement

Problem Statement
Hackathons involve multiple teams competing across several rounds. Manually forming teams, checking team sizes, calculating round-wise scores, and ranking participants can be time-consuming and may lead to errors. The Hackathon Team Formation and Ranking System automates these tasks by validating team composition, calculating total scores, and generating a ranked list. When teams have equal total scores, the system uses the final-round score as a tie-breaker.

2. Objectives

Objectives
To develop a Python program that forms hackathon teams, checks team sizes and student eligibility, calculates total scores across multiple rounds, and ranks teams based on their performance. The program also resolves tied rankings using predefined rules.

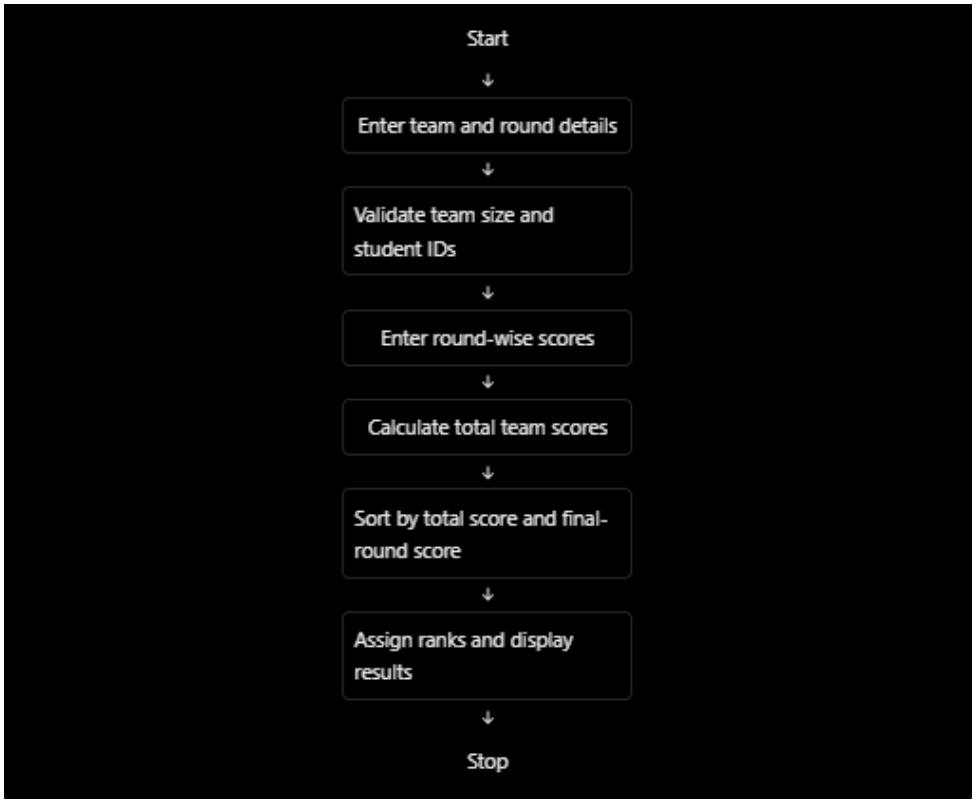
3. Assumptions

Assumptions
1. Each student has a unique student ID and name.
2. Each team must contain at least 2 and at most 4 students.
3. Each student can belong to only one team.
4. Every team participates in the same number of hackathon rounds.
5. Scores are non-negative numbers.
6. The total team score is the sum of scores from all rounds.
7. Teams with higher total scores receive higher ranks.
8. If two teams have equal total scores, the team with the higher final-round score ranks first.
9. If both total and final-round scores are equal, the teams share the same rank.

4. Algorithm

Algorithm
1. Start the program.
2. Enter the number of hackathon teams and the number of rounds.
3. Enter each team's name and number of members.
4. Validate that each team has between 2 and 4 members.
5. Enter each member's student ID and name.
6. Check that no student ID is assigned to multiple teams.
7. Enter each team's score for every round.
8. Calculate the total score for each team.
9. Sort teams by total score in descending order.
10. If total scores are equal, compare final-round scores.
11. Assign the same rank to teams whose total and final-round scores are both equal.
12. Display the final team rankings and scores.
13. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT number of teams
INPUT number of rounds

CREATE an empty list of teams
CREATE an empty list of student IDs

FOR each team:
    INPUT team name
    INPUT number of members

    CHECK team size is between 2 and 4

    INPUT each student's ID and name
    CHECK student ID is not already assigned

    INPUT score for each round
    CALCULATE total score

    STORE team details and scores

SORT teams by:
    1. Total score in descending order
    2. Final-round score in descending order

FOR each ranked team:
    IF total score and final-round score
       equal the previous team's scores:
        ASSIGN the same rank
    ELSE:
        ASSIGN the current position as rank

DISPLAY team names, scores, and ranks

STOP
```

7. Python Source Code / Program

```
print("Hackathon Team Formation and Ranking")

teams = []
n = int(input("Enter number of teams: "))
rounds = int(input("Enter number of rounds: "))

for i in range(n):
    print("\nTeam", i + 1)
    name = input("Enter team name: ")

    size = int(input("Enter number of members (2 to 4): "))
    while size < 2 or size > 4:
        print("Team must have 2 to 4 members.")
        size = int(input("Enter number of members: "))

    members = []
    for j in range(size):
        student_id = input("Enter student ID: ")

        while any(student_id in team["ids"] for team in teams) or student_id in members:
            print("Duplicate student ID! Enter another ID.")
            student_id = input("Enter student ID: ")

        members.append(student_id)

    total = 0
    last_score = 0

    for r in range(rounds):
        score = int(input("Enter score for round " + str(r + 1) + ": "))
        while score < 0:
            print("Score cannot be negative.")
            score = int(input("Enter score again: "))
        total += score
        last_score = score

    teams.append({
        "name": name,
        "ids": members,
        "total": total,
        "last": last_score
    })

teams.sort(key=lambda x: (x["total"], x["last"]), reverse=True)

print("\n--- FINAL RANKINGS ---")

for i in range(n):
    if i > 0 and teams[i]["total"] == teams[i-1]["total"] and teams[i]["last"] ==
    teams[i-1]["last"]:
        rank = teams[i-1]["rank"]
    else:
        rank = i + 1

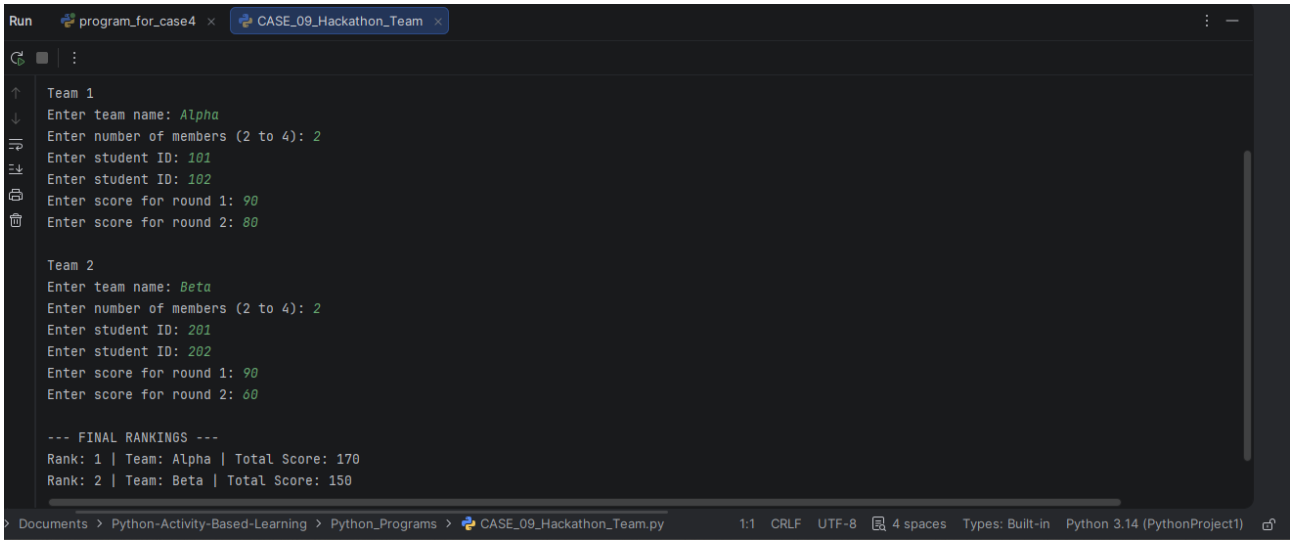
    teams[i]["rank"] = rank
    print("Rank:", rank, "| Team:", teams[i]["name"],
          "| Total Score:", teams[i]["total"])
```

8. Test Cases

.Test Cases

Test case	Input condition	Expected result
1	Team has 3 members	Team size accepted
2	Team has 5 members	Program asks for a valid size
3	Same student ID entered for two teams	Duplicate ID detected
4	Team has a negative score	Program asks for a valid score
5	One team has a higher total score	That team ranks higher
6	Total scores equal, final-round scores differ	Higher final-round score ranks higher
7	Total and final-round scores both equal	Teams share the same rank

9. Output Screenshot



10. Conclusion

This program validates hackathon team composition and student IDs, totals scores across rounds, and ranks teams using the specified tie-break rules. It supports fairer and more organized team evaluation.

CASE 10: University Scholarship Allocation

1. Problem Statement

Problem Statement
Universities receive scholarship applications from many students. Manually checking marks, attendance, and family income can be time-consuming and may cause errors. The University Scholarship Allocation System automates the evaluation process by checking eligibility criteria, preparing a ranked list of eligible applicants, and resolving ties fairly. This helps universities allocate scholarships in an organized and transparent manner.

2. Objectives

Objectives
To develop a Python program that evaluates scholarship applicants based on aggregate marks, subject-wise minimum marks, attendance percentage, and family income. The system identifies eligible students, ranks them according to their academic performance, and resolves ties using predefined rules.

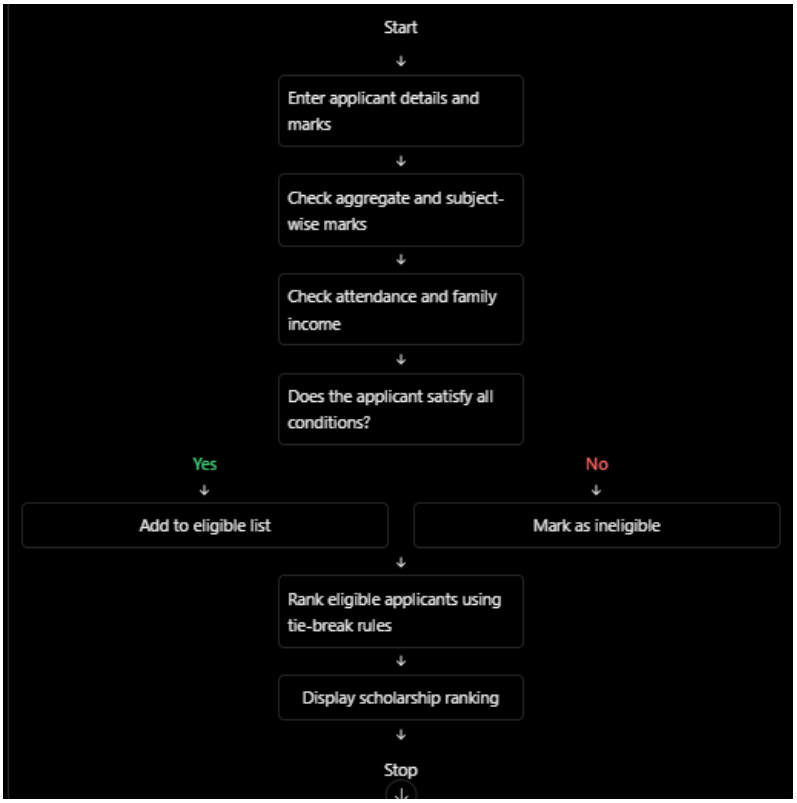
3. Assumptions

Assumptions
1. Each applicant has a unique student ID and name.
2. The minimum aggregate marks required are 75%.
3. Each subject must have at least 60 marks out of 100.
4. The minimum attendance requirement is 75%.
5. The maximum annual family income allowed is ₹2,50,000.
6. Applicants must satisfy all four eligibility conditions to qualify.
7. Eligible applicants are ranked by aggregate marks in descending order.
8. If aggregate marks are equal, higher attendance receives the higher rank.
9. If both aggregate marks and attendance are equal, the applicant with lower family income receives the higher rank.
10. The program uses these assumed criteria for educational purposes; actual scholarship rules may differ.

4. Algorithm

Algorithm
1. Start the program.
2. Enter the number of scholarship applicants.
3. For each applicant, enter their student ID, name, aggregate percentage, subject marks, attendance percentage, and annual family income.
4. Check whether the aggregate marks are at least 75%.
5. Check whether every subject mark is at least 60.
6. Check whether attendance is at least 75%.
7. Check whether annual family income is ₹2,50,000 or less.
8. Add applicants who satisfy all conditions to the eligible list.
9. Sort eligible applicants by aggregate marks, attendance, and family income.
10. Assign ranks according to the predefined tie-breaking rules.
11. Display the eligible applicants, their ranks, and the total number of eligible students.
12. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT number of applicants

CREATE an empty eligible list

FOR each applicant:
    INPUT student ID and name
    INPUT aggregate percentage
    INPUT marks of each subject
    INPUT attendance percentage
    INPUT annual family income

    IF aggregate >= 75
       AND every subject mark >= 60
       AND attendance >= 75
       AND income <= 250000:
           ADD applicant to eligible list
    ELSE:
           MARK applicant as ineligible

SORT eligible applicants by:
    1. Aggregate marks (highest first)
    2. Attendance (highest first)
    3. Family income (lowest first)

ASSIGN ranks using the tie-breaking rules

DISPLAY eligible applicants and ranks

STOP
```

7. Python Source Code / Program

```
print("University Scholarship Allocation")

students = []

n = int(input("Enter number of students: "))

for i in range(n):
    print("\nStudent", i + 1)

    name = input("Enter student name: ")
    aggregate = float(input("Enter aggregate marks (%): "))
    maths = int(input("Enter Mathematics marks: "))
    science = int(input("Enter Science marks: "))
    attendance = float(input("Enter attendance (%): "))
    income = int(input("Enter family income: "))

    if (aggregate >= 75 and maths >= 60
            and science >= 60 and attendance >= 75
            and income <= 250000):
        print("Eligible for scholarship")
        students.append([name, aggregate, attendance, income])
    else:
        print("Not eligible")

# Rank students
students.sort(key=lambda x: (-x[1], -x[2], x[3]))

print("\n--- SCHOLARSHIP RANKING ---")

for i in range(len(students)):
    print("Rank:", i + 1)
    print("Name:", students[i][0])
    print("Aggregate:", students[i][1], "%")
    print("Attendance:", students[i][2], "%")
    print("Family income: Rs.", students[i][3])

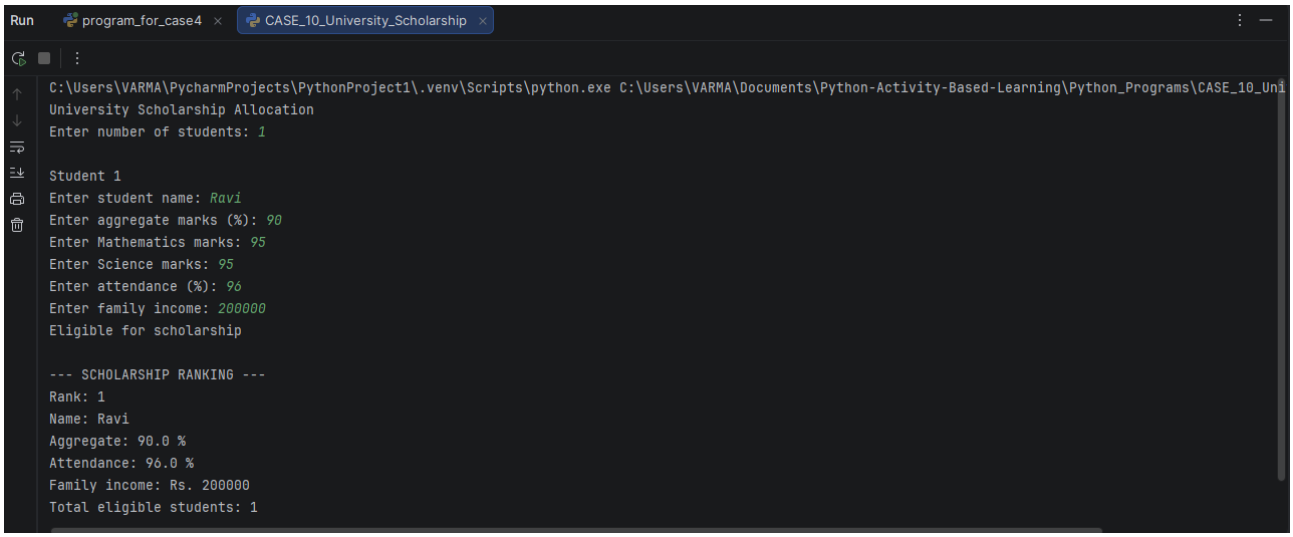
print("Total eligible students:", len(students))
```

8. Test Cases

Test Cases

Test case	Input condition	Expected result
1	All eligibility conditions satisfied	Eligible
2	Aggregate below 75%	Ineligible
3	Any subject mark below 60	Ineligible
4	Attendance below 75%	Ineligible
5	Family income above ₹2,50,000	Ineligible
6	Equal aggregate, different attendance	Higher attendance ranks first
7	Equal aggregate and attendance	Lower family income ranks first

9. Output Screenshot



10. Conclusion

This program checks scholarship applicants against academic, attendance, and income conditions, then ranks eligible students using the stated criteria. It makes the evaluation process more systematic and transparent.